

OOPS [Object Oriented Programming]

- APNA CLG

Page No.:

Date:

→ Definitions + eg

* Classes & Objects

- Objects are entities in the real world
- class is like blueprint of these entities -
or
Group of objects

Syntax

```
class Teacher {  
    Prop  
    method  
};
```

* Access Modifiers default

- (i) Private - data & methods accessible inside class.
- (ii) Public - data & methods accessible to everyone
- (iii) Protected - data & methods accessible inside class to its derived class.

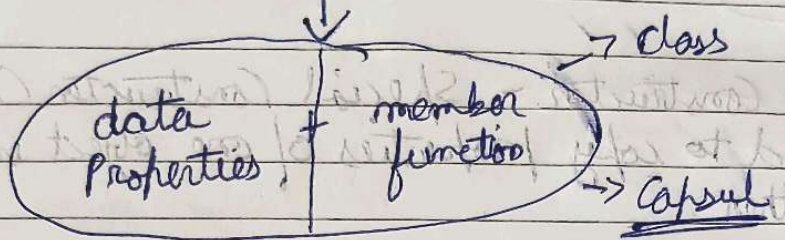
Inheritance

* Encapsulation :-

Encapsulation is wrapping up of data & member functions into a single unit called class.

* OOPS Pillars 4 most :-

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism



* Helps in data Hiding → Private

⇒ Constructor → (Allocation of memory)
 Special method invoked automatically at time of object creation.
 Used for initiation.

- Same name as class
- Constructor doesn't have a return type.
- Only called once (automatically) at object creation.
- Memory allocation happens when constructor is called.

* Three types of Constructor -

- Non-parameterized -
- Parameterized -
- Copy const.

Polymorphism
 Eg

* The phenomena in which same is some best type can be diff is called constructor Overloading.
 like multiple constructor can be created -

M T W T F S
Page No.:
Date: VOL

* This is a special pointer in C++ that points to the current objects.

this $\rightarrow$ prop is same as *(this) - prop
 $\downarrow$ $\downarrow$
Easy Pointers

(iii) Copy Constructor - Special Constructor (default) used to copy properties of one object into another.

* Two Copy Const type:-

- (i) Shallow Copy
- (ii) Deep Copy

* A Shallow copy of an object copies all of the member values from one object to another.

* A Deep copy, on the other hand, not only copies the member values but also makes copies of any dynamically allocated memory that the members point to.

* Destructor - opposite of Constructor (Deallocate of memory).

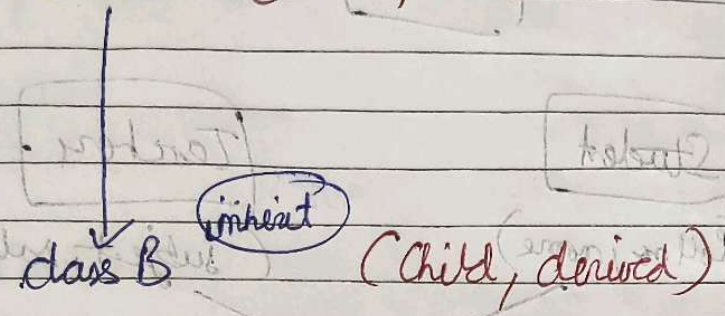
Allocate - new
Deallocate - delete

used

* Inheritance - Code reusability :-

When properties & member functions of base class are passed on the derived class.

Class A (Parent, Base)

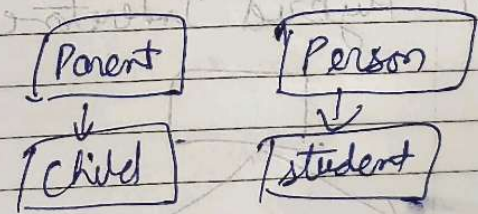


* Mode of Inheritance

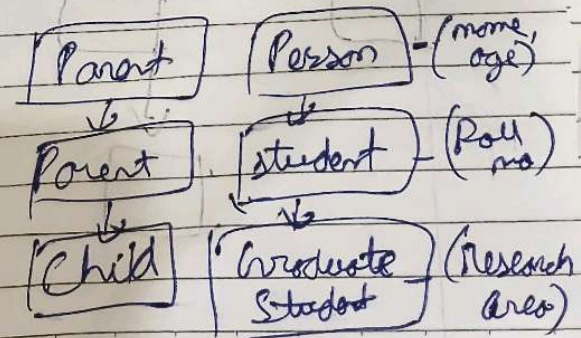
Parent Base class	Derived class	Derived class	Derived class
	Pvt Mode	Protect Mode	Public Mode
Private	Not inherit	Not	Not
Protected	Private	Protected	Protected
Public	Private	Protected	Public

* Types of Inheritance

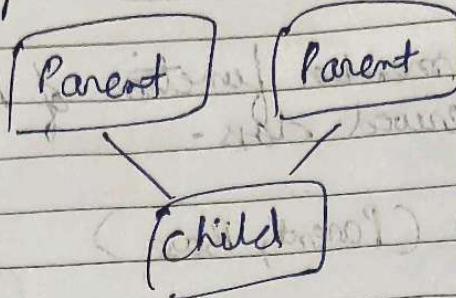
(i) Single Inheritance



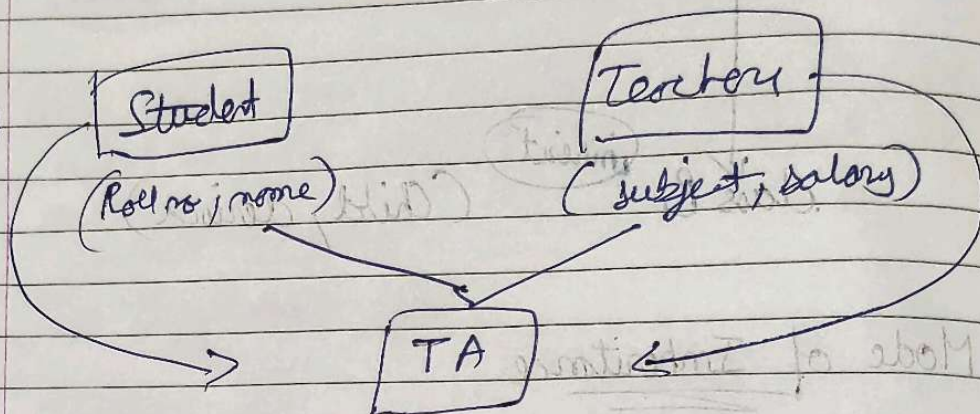
(ii) Multilevel Inheritance



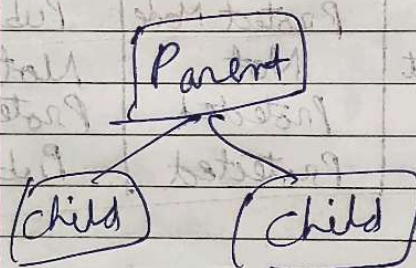
(iii) Multiple Inheritance



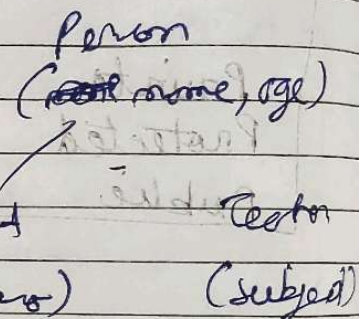
Eg



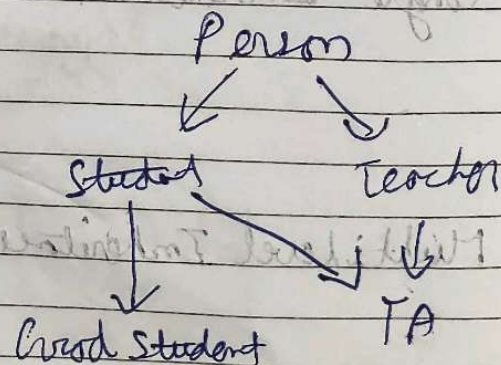
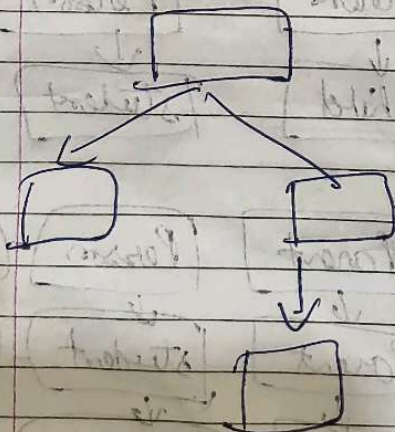
(iv) Hierarchical Inheritance



Eg



(v) Hybrid Inheritance



multiple — Forms

M	T	W	T	F	S	S
Page No.:						YOUVA
Date:						

37. Polymorphism — Polymorphism is the ability of objects to take one different forms or behave in different ways depending on the context in which they are used.

* Types of Polymorphism

- i) Compile Time
 - ii) Run Time
- 1) name ✓ Param x
- 2) name ✓ Parameter x

Eg:- Constructor Overloading

* Compile Time Polymorphism (Eg)

• Function Overloading

Class {
 func (↓) Parameter } diff
 func (↓) Parameter } no. of Para
 func (↓) Parameter } type of Para

};

• Operator Overloading

int y = 10 }
 int x = y } (x = 10)
 string b = "abc" }
 string a = b } a = "abc"

Overloading

class

Inheritance

Page No.:

Date:

* Run-time Polymorphism

• Function Overriding

Parent & Child both contain the same func with the different implementation. The parent class function is said to be overridden.

• Virtual Functions

A virtual func is a member func that you expect to be redefined in derived class.

• Virtual functions are Dynamic in nature

• Defined by the keyword "virtual" inside a base class and are always declared with a base class & overridden in a child class.

• A virtual function is called during Runtime.

4% Abstraction -

Hiding all ^{sensitive} unnecessary details & showing only the important parts.

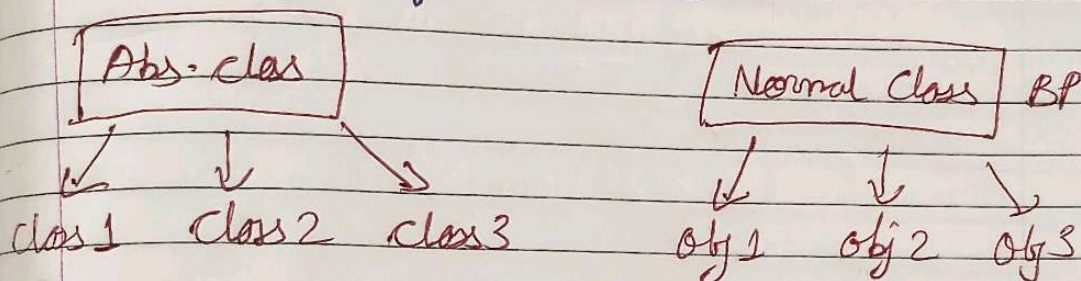
Access modifiers / Access specifiers

Private

Protected

Public

- Using Abstract classes.
- Abstract classes are used to provide a base class from which other classes can be derived.
- They cannot be instantiated & are ~~not~~ meant to be instantiated.
- Abstract classes are typically used to define an interface for derived class.



* Static Keyword :-

- Static Variables :-

Variables declared as static functions are created & initialised once for the lifetime of the program.

// in function

Static variables in a class are created & initialised once. They are shared by all the object of the class.

// in class